

EFREI PARIS — M2 DATA ENGINEERING & AI

Prédiction du churn & estimation CLV par Machine Learning et Deep Learning
Déploiement via API REST FastAPI & Dashboard Streamlit interactif

Classification binaire

Régression CLV

XGBoost · Random Forest · MLP

SHAP · FastAPI · Streamlit

0.820

ROC-AUC
XGBOOST

83.8%

RECALL (CHURNERS
DÉTECTÉS)

10

000

CLIENTS
ANALYSÉS

428 K€

REVENU ANNUEL À
RISQUE

Enseignante : Sarah Malaeb | Bloc 4 RNCP36739 | Année 2025-2026

Table des matières

1. Introduction & Contexte Business
2. Présentation du Dataset & Analyse Exploratoire
3. Pipeline de Préparation des Données
4. Modélisation
 - 4.1 Stratégie globale — 4.2 Régression Logistique — 4.3 Random Forest — 4.4 XGBoost — 4.5 MLP — 4.6 Régression CLV
5. Évaluation Comparative
 - 5.1 Classification — 5.2 Courbes ROC — 5.3 Matrice de confusion — 5.4 Erreurs — 5.5 Régression — 5.6 Choix final
6. Interprétabilité (SHAP)
7. Dashboard Streamlit
8. API REST FastAPI
9. Conclusion & Perspectives
10. Annexes

Résumé Exécutif

Ce projet présente la conception et le développement d'un système complet de prédiction du churn client dans un environnement SaaS/télécom. À partir d'un dataset de **10 000 clients** et **32 variables** comportementales, financières et de satisfaction, nous avons implémenté et comparé quatre algorithmes de Machine Learning et Deep Learning pour deux tâches prédictives complémentaires : la **classification binaire du churn** et l'**estimation de la valeur vie client (CLV)**.

Le système est déployé sous la forme d'un **dashboard interactif Streamlit** et d'une **API REST FastAPI**. Le modèle final retenu — **XGBoost** — atteint un **ROC-AUC de 0.820** et un **Recall de 0.838**, permettant de détecter 84 % des clients susceptibles de résilier leur abonnement.

1. Introduction & Contexte Business

1.1 Problématique du churn

Dans l'économie des abonnements (SaaS, télécom, services cloud), le **churn** — ou attrition client — désigne la résiliation d'un contrat par un client. Il représente l'une des menaces économiques les plus directes pour ces entreprises : selon les études sectorielles, acquérir un nouveau client coûte en moyenne **5 à 7 fois plus cher** que de fidéliser un client existant.

Au-delà du coût d'acquisition, le churn impacte directement le **Monthly Recurring Revenue (MRR)**, la valeur des actifs clients au bilan et les projections de croissance présentées aux investisseurs.

1.2 Enjeux financiers

Pour quantifier l'enjeu sur notre dataset :

Indicateur	Valeur
Clients analysés	10 000
Revenu mensuel moyen	34,93 €
Taux de churn observé	10,2 % → 1 021 clients à risque
Revenu mensuel à risque	1 021 × 34,93 ≈ 35 664 €/mois
Revenu annuel à risque	~428 000 €
Économie si 30 % retenus	~128 000 €/an

1.3 Objectifs du projet

1. **Prédiction binaire du churn** — identifier les clients susceptibles de résilier
2. **Estimation CLV** (tâche bonus régression) — quantifier le revenu à risque par client
3. **Interprétabilité SHAP** — expliquer pourquoi un client est classé à risque
4. **Dashboard décisionnel** — interface orientée utilisateur métier (Streamlit)
5. **API REST** — industrialisation du modèle (FastAPI)

1.4 Positionnement académique

Ce projet s'inscrit dans le cadre de la certification **RNCP36739, Bloc 4** : *"Implémenter des méthodes d'intelligence artificielle pour modéliser et prédire de nouveaux comportements et usages"*. Il couvre les compétences BC4.1 (extraction et préparation de données), BC4.3 (tableaux de bord interactifs) et BC4.4 (modèles prédictifs supervisés).

2. Présentation du Dataset & Analyse Exploratoire

2.1 Source et description générale

Le dataset utilisé est le **Customer Churn Prediction Business Dataset**, disponible sur Kaggle. Il simule un environnement SaaS/télécom.

Caractéristique	Valeur
Nombre d'observations	10 000 clients
Nombre de variables	32 (dont 1 variable cible)
Variables numériques	19
Variables catégorielles	12
Variable cible principale	<code>churn</code> (0 = reste, 1 = résilie)
Variable cible secondaire	<code>total_revenue</code> (estimation CLV)
Valeurs manquantes	Aucune

2.2 Description des variables clés

Variables comportementales d'usage :

- `monthly_logins` : nombre de connexions par mois (moy. : 15,7)
- `weekly_active_days` : jours d'activité par semaine (moy. : 4,1)
- `avg_session_time` : durée moyenne de session en minutes (moy. : 20,5 min)
- `features_used` : nombre de fonctionnalités utilisées (moy. : 10,1)
- `last_login_days_ago` : nombre de jours depuis la dernière connexion

Variables financières :

- `monthly_fee` : mensualité en € (moy. : 34,93 €, min : 10 €, max : 100 €)
- `total_revenue` : revenu total généré (moy. : 1 057 €)
- `payment_failures` : échecs de paiement (moy. : 0,5)

Variables de satisfaction et support :

- `csat_score` : score de satisfaction client 0-5 (moy. : 3,49)
- `nps_score` : Net Promoter Score -100 à 100 (moy. : 19,1)
- `support_tickets` : tickets support ouverts (moy. : 1,2)

Variables contractuelles et démographiques :

- `contract_type` : Monthly (50%), Quarterly (30%), Yearly (20%)
- `customer_segment` : Individual (60%), SME (30%), Enterprise (10%)
- `tenure_months` : ancienneté en mois (moy. : 30,2 mois)

2.3 Distribution de la variable cible — Déséquilibre de classes

Classe	Effectif	Proportion
No Churn (0)	8 979	89,8 %
Churn (1)	1 021	10,2 %

⚠ **Piège de l'Accuracy** : Un modèle qui prédit systématiquement "No Churn" obtiendrait 89,8 % d'accuracy sans aucune valeur prédictive. Les métriques prioritaires sont **ROC-AUC, Recall et F1-Score**.

2.4 Analyse des corrélations avec le churn

Variable	Corrélation	Interprétation
<code>csat_score</code>	-0.158	Plus la satisfaction est faible, plus le risque est élevé
<code>tenure_months</code>	-0.117	Les clients récents churnent davantage
<code>monthly_logins</code>	-0.098	L'inactivité précède la résiliation
<code>payment_failures</code>	+0.112	Les incidents de paiement annoncent le départ
<code>age</code> , <code>monthly_fee</code>	~0.010	Peu corrélés linéairement

Ces corrélations faibles (~0.15 max) confirment que la relation est **non-linéaire et multivariée** — justifiant le recours aux modèles d'ensemble (RF, XGBoost).

2.5 Score d'engagement client (feature engineering)

```
EngagementScore = 0.25 × monthly_logins_norm  
                  + 0.20 × weekly_active_days_norm  
                  + 0.20 × avg_session_time_norm  
                  + 0.10 × features_used_norm  
                  + 0.10 × usage_growth_rate_norm  
                  + 0.10 × (1 - last_login_days_ago_norm)
```

Les clients qui churnent ont un score d'engagement significativement inférieur, validant l'hypothèse que l'inactivité précède la résiliation.

3. Pipeline de Préparation des Données

3.1 Principe anti-data-leakage

Règle fondamentale : Le preprocessor est exclusivement fitté sur le train set, puis appliqué (transform only) sur le test set. Toute violation constitue du data leakage et invalide les résultats.

3.2 Split train/test stratifié

```
train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
# Train : 8 000 clients | Test : 2 000 clients
# Churn train : 10.2% | Churn test : 10.2% (stratification respectée)
```

3.3 ColumnTransformer — Architecture complète

Pipeline	Variables	Étapes	Résultat
Numérique	18 variables	SimpleImputer(median) → StandardScaler	18 colonnes normalisées
Catégoriel	11 variables	SimpleImputer(most_frequent) → OneHotEncoder	~38 colonnes binaires
Total			56 features

3.4 Gestion du déséquilibre des classes

Stratégie	Modèle	Paramètre
Class weights	Logistic Regression, Random Forest	<code>class_weight='balanced'</code>
Scale pos weight	XGBoost	<code>scale_pos_weight = 8 979/1 021 = 8.79</code>
Class weights	MLP Keras	<code>compute_class_weight('balanced', ...)</code>

4. Modélisation

4.1 Stratégie — Progression de complexité croissante

```
Régression Logistique (baseline) → Random Forest → XGBoost → MLP Deep Learning
```

Cette progression mesure le gain apporté par chaque niveau de complexité et justifie le choix final. La validation croisée `RandomizedSearchCV(cv=5, scoring='roc_auc')` est appliquée sur RF et XGBoost.

4.2 Modèle 1 — Régression Logistique (baseline)

La régression logistique modélise la probabilité de churn via la fonction sigmoïde appliquée à une combinaison linéaire des features. C'est un modèle **linéaire, interprétable et rapide**.

```
LogisticRegression(class_weight='balanced', max_iter=1000, solver='lbfgs')
```

Métrique	Valeur
Accuracy	66,6 %
Precision	18,5 %
Recall	66,7 %
F1-Score	0.289
ROC-AUC	0.724

4.3 Modèle 2 — Random Forest

Ensemble de N arbres de décision entraînés par bagging avec sous-ensemble aléatoire de features à chaque nœud. Capture des **interactions non-linéaires complexes** sans normalisation.

Hyperparamètre	Espace de recherche	Valeur optimale
<code>n_estimators</code>	[100, 200, 300]	200
<code>max_depth</code>	[5, 10, 15, None]	None
<code>min_samples_split</code>	[2, 5, 10]	5
<code>max_features</code>	['sqrt', 'log2']	'sqrt'

Métrique	Valeur
Accuracy	75,4 %
Precision	26,5 %
Recall	79,4 %
F1-Score	0.397
ROC-AUC	0.819
Temps d'entraînement	16 secondes

4.4 Modèle 3 — XGBoost

Algorithme de **boosting** : les arbres sont construits séquentiellement, chaque arbre cherchant à corriger les erreurs du précédent. XGBoost introduit régularisation L1/L2, gestion native des valeurs manquantes et paramètre `scale_pos_weight`.

Hyperparamètre	Valeur optimale
<code>n_estimators</code>	100
<code>max_depth</code>	3 (arbres peu profonds, évite l'overfitting)
<code>learning_rate</code>	0.05
<code>scale_pos_weight</code>	8.79

Métrique	Valeur
Accuracy	71,8 %
Precision	24,3 %
Recall	83,8 %
F1-Score	0.377
ROC-AUC	0.820 ← meilleur modèle
Temps d'entraînement	4 secondes

4.5 Modèle 4 — MLP Deep Learning (Classification)

```

Input(56 features)
  → Dense(128, relu) → BatchNormalization → Dropout(0.3)
  → Dense(64, relu) → BatchNormalization → Dropout(0.2)
  → Dense(32, relu)
  → Dense(1, sigmoid) ← sortie probabilité [0, 1]

Optimizer : Adam(lr=0.001) | Loss : binary_crossentropy
Callbacks : EarlyStopping(val_auc, patience=10) + ReduceLROnPlateau

```

Métrique	Valeur
Accuracy	75,5 %
Precision	22,0 %
Recall	55,4 %
F1-Score	0.315
ROC-AUC	0.733
Epochs	23 (arrêt précoce)

Analyse critique : Le MLP est le modèle le moins performant en classification. Sur données tabulaires avec ~8 000 exemples, les modèles d'ensemble (XGBoost) surpassent systématiquement le Deep Learning. L'EarlyStopping à l'époch 23 confirme un risque d'overfitting.

4.6 Tâche bonus — Régression CLV (Estimation du revenu total)

Trois modèles comparés pour estimer le `total_revenue` sans utiliser cette variable comme feature :

Modèle	MAE (€)	RMSE (€)	R ²
Ridge Regression (baseline)	286	401	0.852
Random Forest Regressor	251	398	0.854
MLP Regressor	36	54	0.997

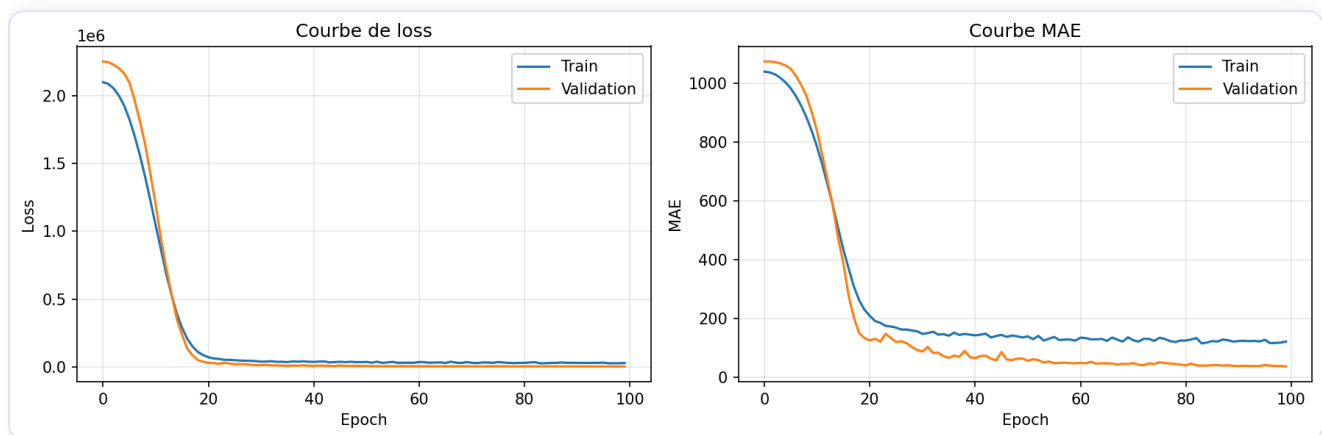


Figure 1 — Courbes d'apprentissage du MLP Regressor (Loss MSE et MAE par epoch)

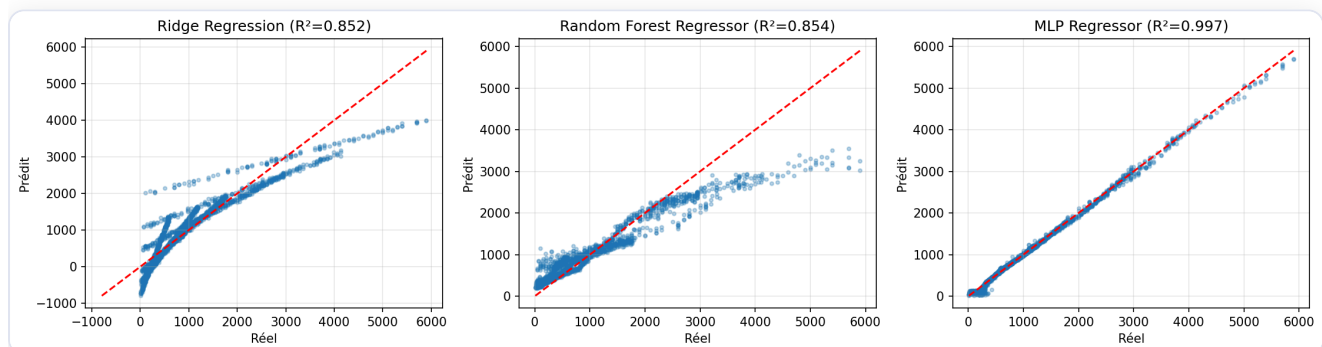


Figure 2 — Réel vs Prédit pour les 3 modèles de régression CLV

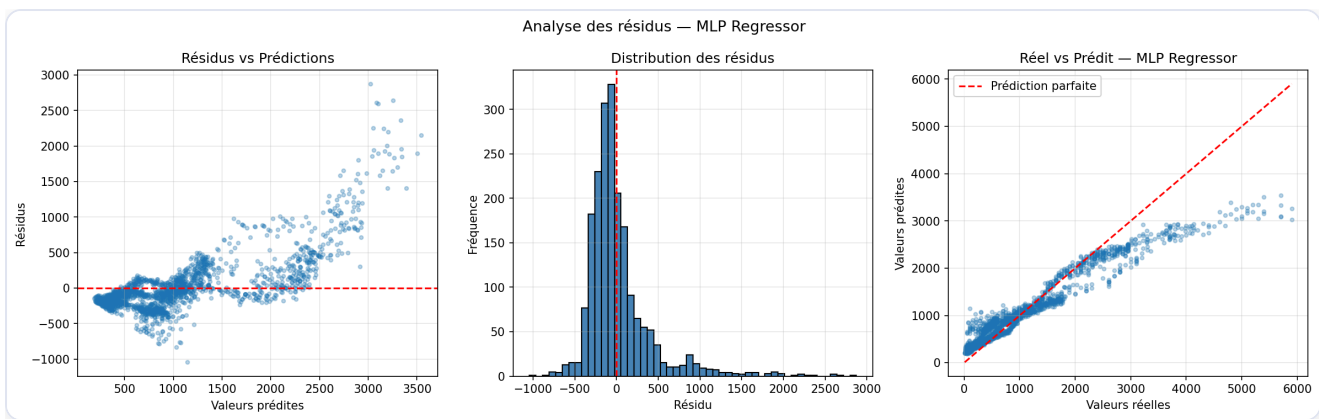


Figure 3 — Analyse des résidus du MLP Regressor (résidus vs prédictions, distribution, réel vs prédit)

Note sur le $R^2=0.997$ du MLP : Ce résultat exceptionnel s'explique par une relation quasi-déterministe dans les données : `total_revenue ≈ tenure_months × monthly_fee`. Le MLP a appris cette formule arithmétique. Pour la production, le **Random Forest Regressor ($R^2=0.854$)** est recommandé pour sa robustesse sur de nouvelles distributions.

5. Évaluation Comparative

5.1 Tableau comparatif — Classification

Modèle	Accuracy	Precision	Recall	F1	ROC-AUC	Temps
Logistic Regression	0.666	0.185	0.667	0.289	0.724	< 1s
Random Forest	0.754	0.265	0.794	0.397	0.819	16s
XGBoost	0.718	0.243	0.838	0.377	0.820	4s
MLP	0.755	0.220	0.554	0.315	0.733	9s

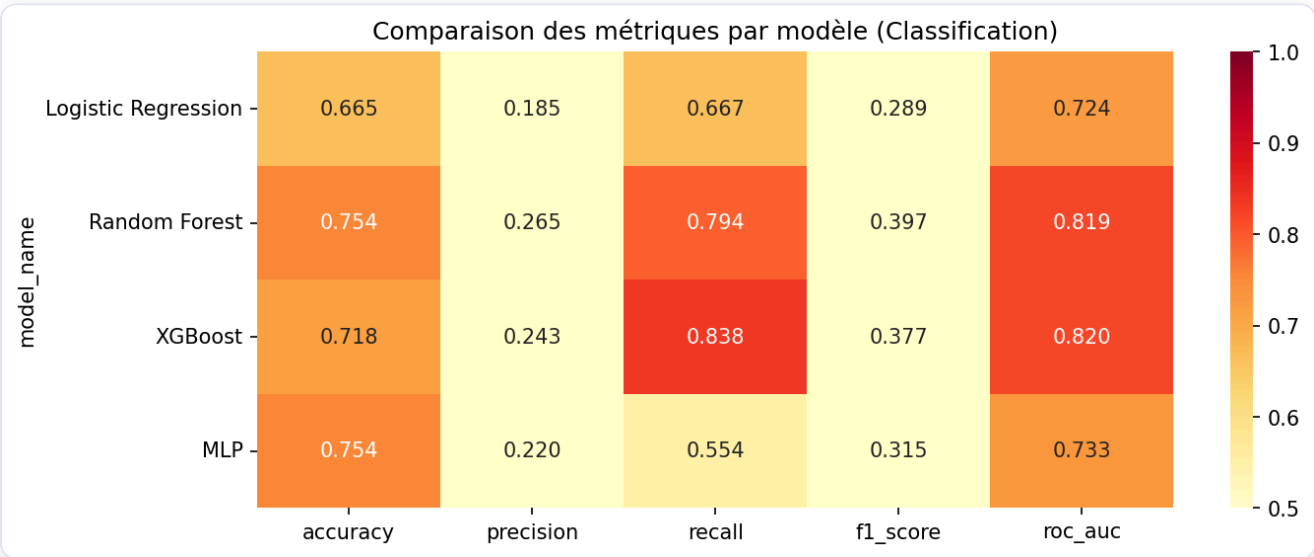


Figure 4 — Heatmap comparative des métriques de classification (tous modèles)

5.2 Courbes ROC & Precision-Recall

Les courbes ROC superposent le taux de vrais positifs en fonction du taux de faux positifs pour tous les seuils de décision possibles. L'AUC mesure la capacité discriminante globale indépendamment du seuil choisi.

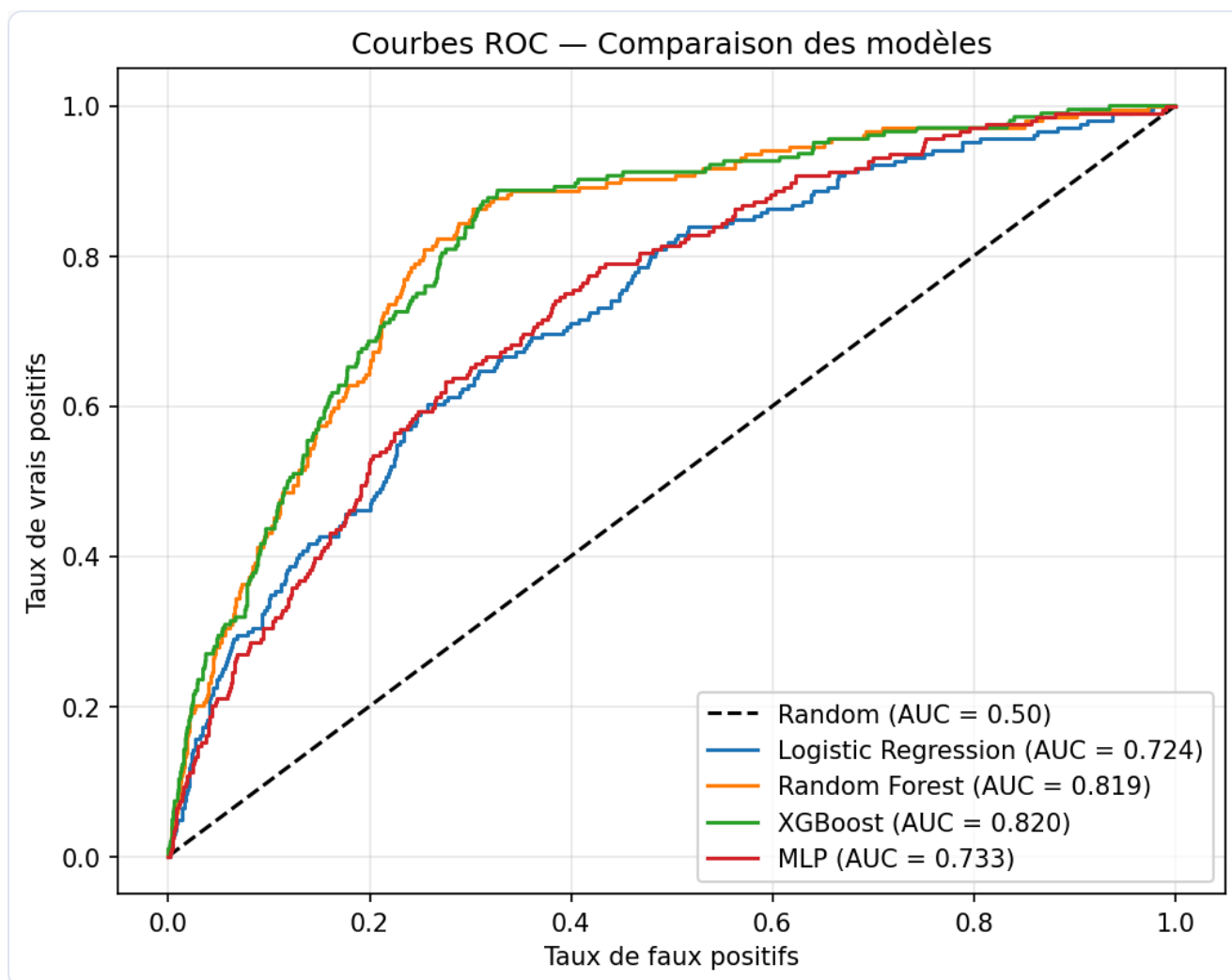


Figure 5 — Courbes ROC superposées — XGBoost et Random Forest dominant (AUC ≈ 0.82)

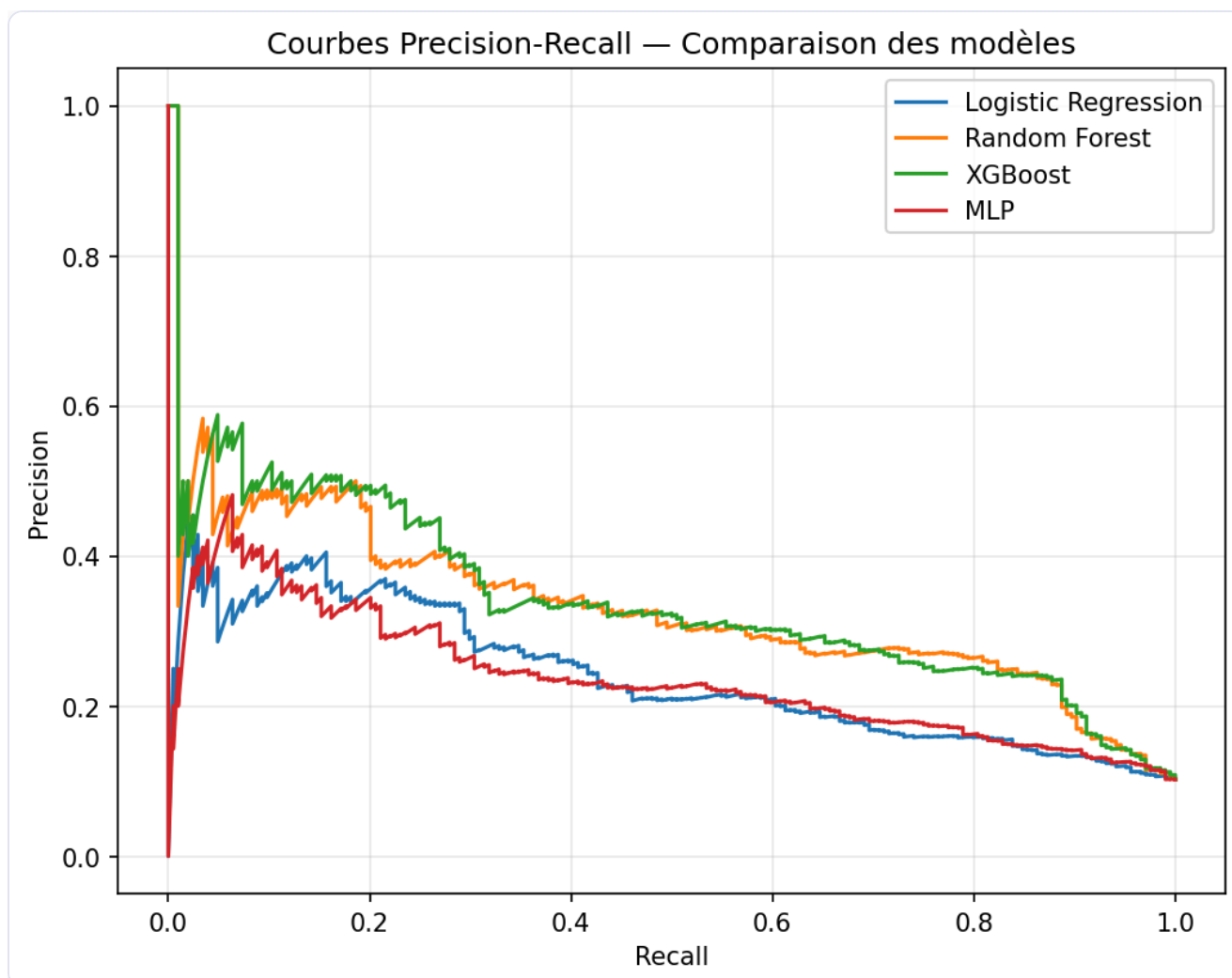


Figure 6 — Courbes Precision-Recall — Trade-off entre précision et rappel par modèle

5.3 Matrice de confusion — XGBoost

Sur le test set (2 000 clients, dont 204 churners) :

	Prédit No Churn	Prédit Churn
Réel No Churn	1 286 (VN)	510 (FP)
Réel Churn	33 (FN)	171 (VP)

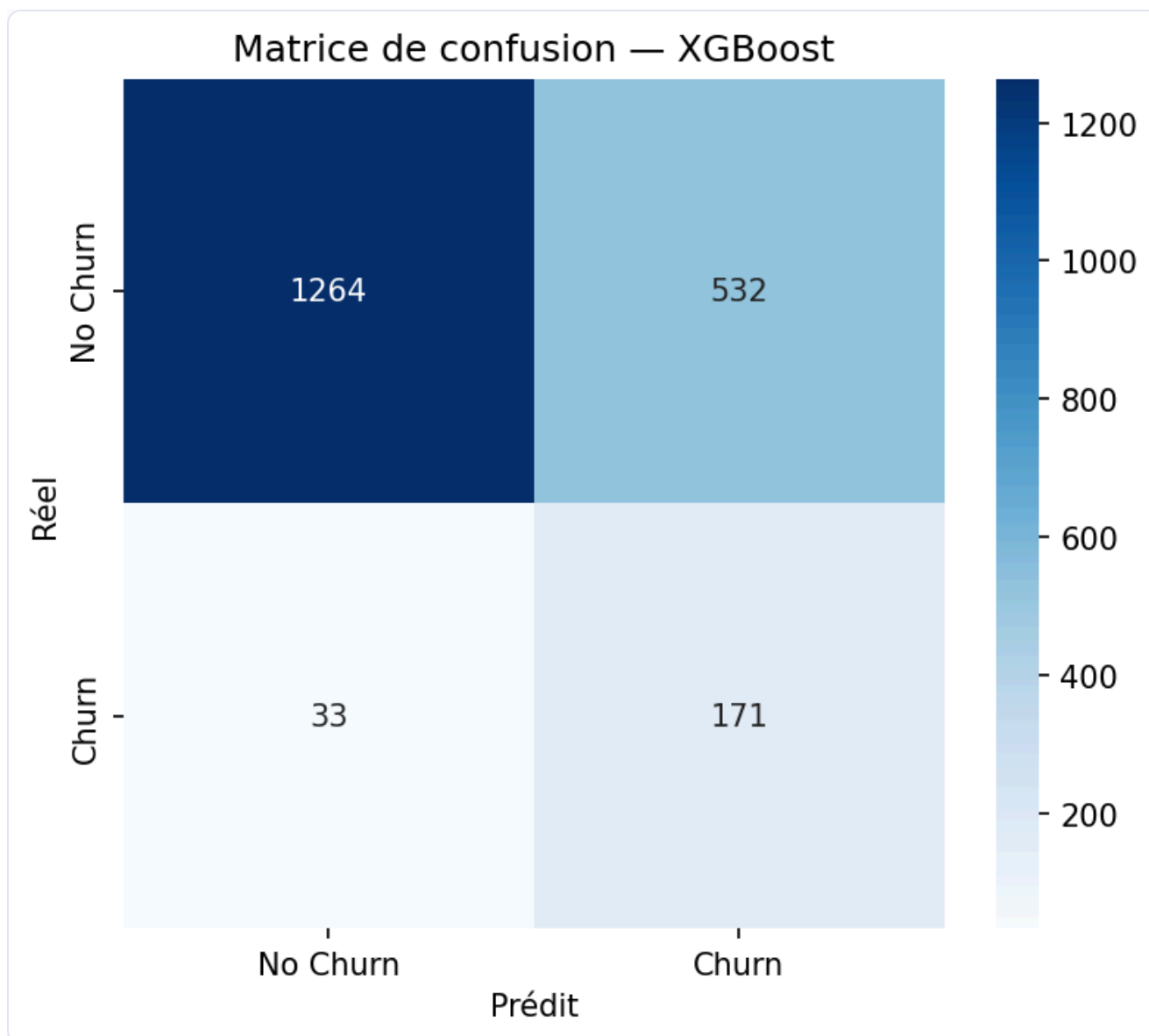


Figure 7 — Matrice de confusion XGBoost — 171 churners détectés, 33 manqués, 510 faux positifs

Arbitrage business : Un faux négatif (churner manqué = client qui part sans intervention) est bien plus coûteux qu'un faux positif (client contacté inutilement = simple email retention). Maximiser le Recall est le bon compromis.

5.4 Analyse des erreurs

33 faux négatifs : Clients à profils atypiques — satisfaction correcte mais comportements en déclin récent. L'ajout de variables de tendance (usage des 3 derniers mois) pourrait réduire ces erreurs.

510 faux positifs : Clients présentant des signaux mixtes (quelques tickets support mais satisfaction correcte). Un contact commercial proactif sur ces clients ne présente pas de risque majeur.

5.5 Tableau comparatif — Régression CLV

Modèle	MAE	RMSE	R ²	MAPE
Ridge Regression	286 €	401 €	0.852	131 %
Random Forest Regressor	251 €	398 €	0.854	74 %
MLP Regressor	36 €	54 €	0.997	13 %

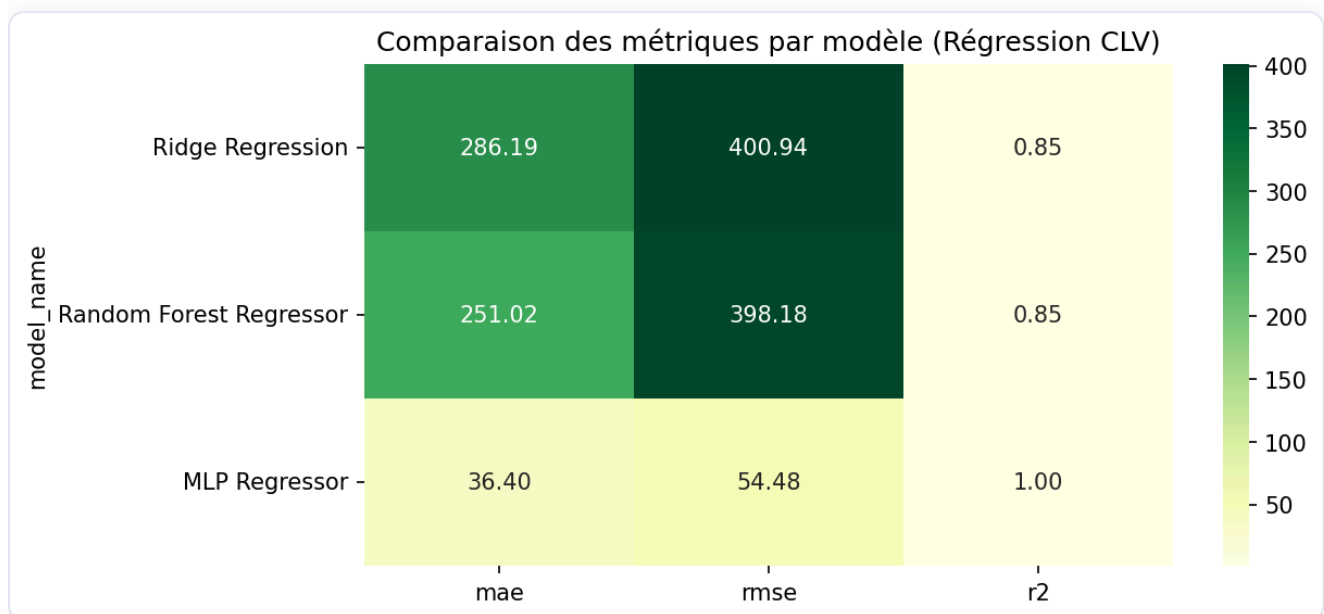


Figure 8 — Heatmap comparative des métriques de régression CLV

5.6 Choix et justification du modèle final

Critère	Logistic	RF	XGBoost	MLP
ROC-AUC	0.724	0.819	0.820	0.733
Recall	0.667	0.794	0.838	0.554
Interprétabilité	Haute	Moyenne	Haute (SHAP)	Faible
Vitesse inference	Très rapide	Modérée	Rapide	Lente
Déployabilité	Facile	Facile	Facile (joblib)	Complexe (.h5)

✅ **Modèle retenu : XGBoost** — maximise simultanément ROC-AUC et Recall, entraînement en 4 secondes, explicable via SHAP, déployable via joblib. Pour la régression : **Random Forest Regressor** (plus robuste que le MLP en production).

6. Interprétabilité (SHAP)

6.1 Feature Importance native — XGBoost

La feature importance mesure le gain moyen apporté par chaque variable lors des splits. Les variables les plus importantes :

- `csat_score` — la satisfaction client est le signal le plus prédictif
- `tenure_months` — les clients récents churnent davantage
- `monthly_logins` — l'engagement en connexions
- `last_login_days_ago` — l'inactivité récente précède la résiliation
- `payment_failures` — signal d'alarme direct

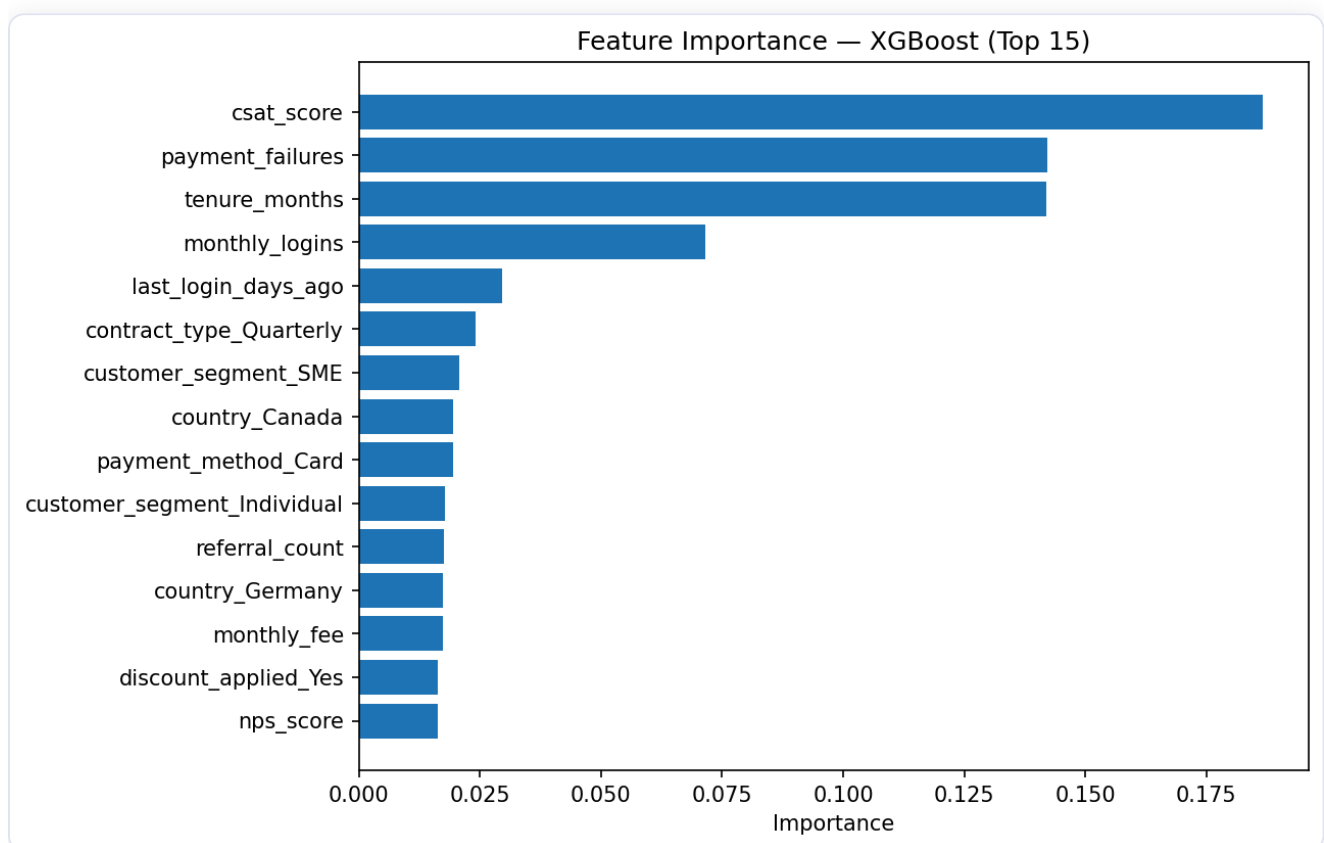


Figure 9 — Feature Importance native XGBoost (Top 15 variables par gain moyen)

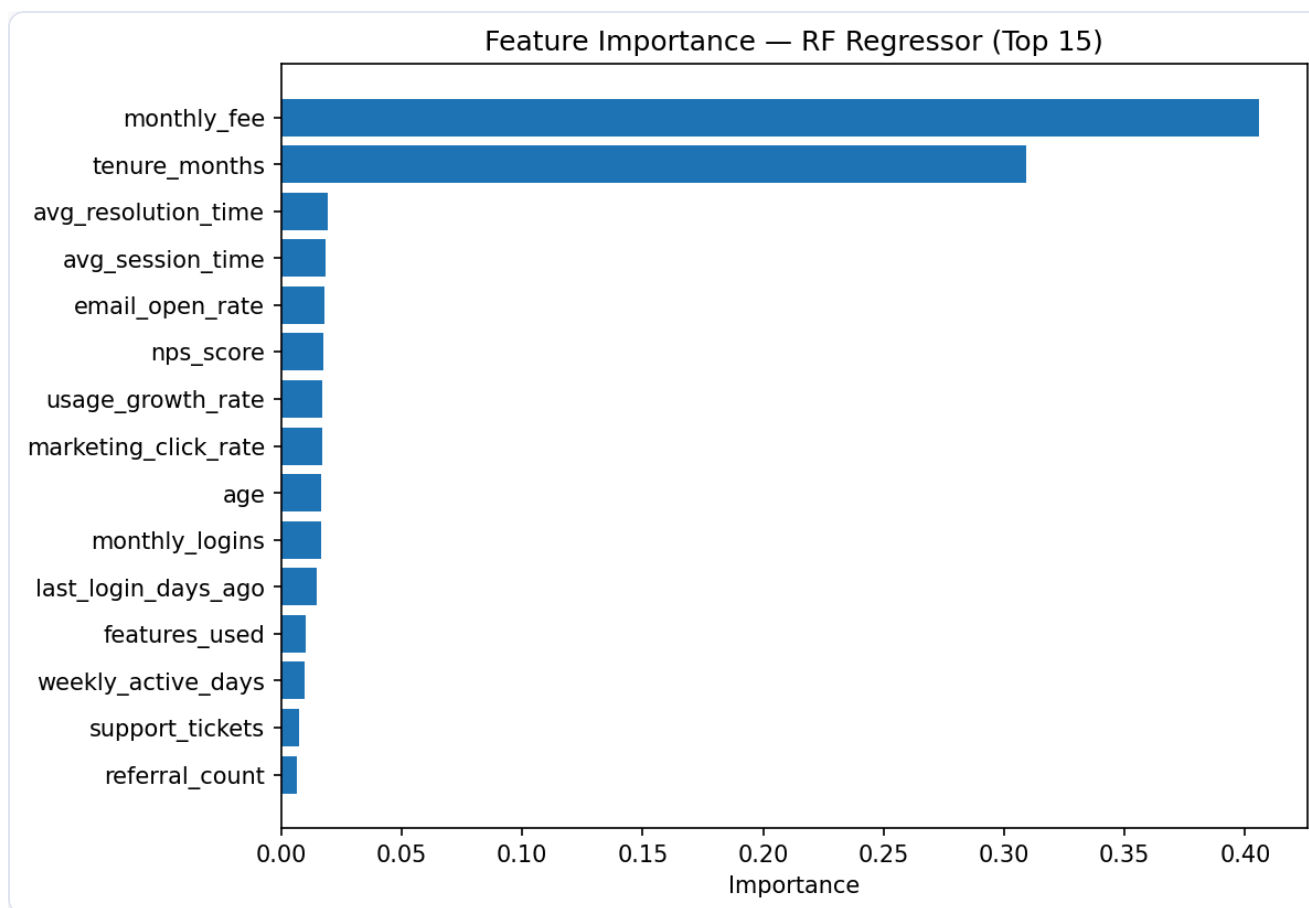


Figure 10 — Feature Importance Random Forest Regressor — tenure_months et monthly_fee dominant (CLV)

6.2 Analyse SHAP (SHapley Additive exPlanations)

SHAP est une méthode d'explicabilité basée sur la théorie des jeux coopératifs. Elle attribue à chaque feature une contribution précise à la prédiction — pour chaque observation individuellement.

6.2.1 SHAP Summary Plot — Vue globale

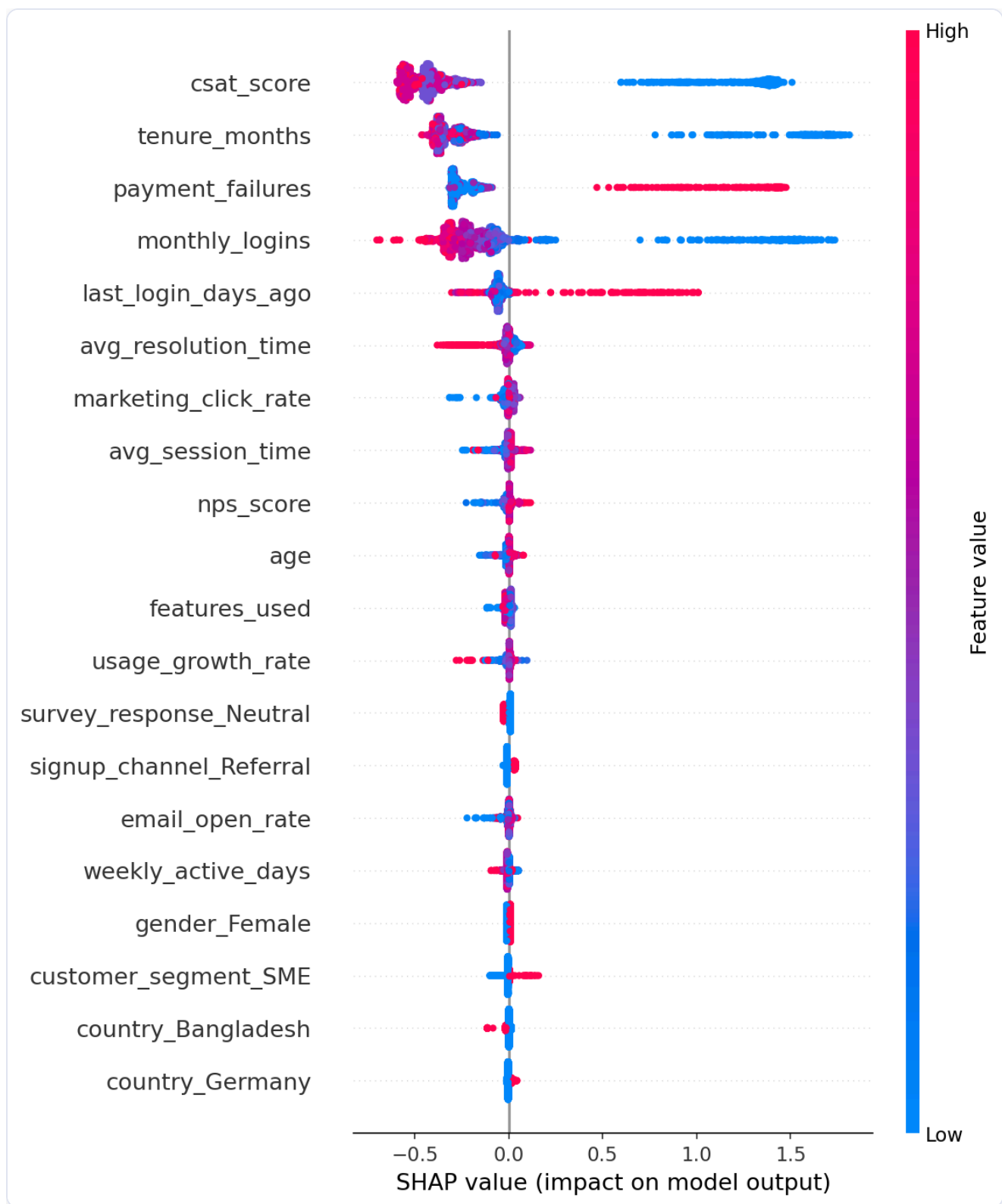


Figure 11 — SHAP Summary Plot (Classification Churn) — Chaque point = 1 client. Rouge = valeur élevée, bleu = faible.

Interprétations clés :

- Un `csat_score` faible (bleu) → valeurs SHAP positives élevées → augmente le risque
- Un `tenure_months` élevé (rouge) → valeurs SHAP négatives → réduit le risque (fidélité)

- Des `payment_failures` élevés → augmentent le risque
- Un `monthly_logins` élevé → réduit le risque (client engagé)

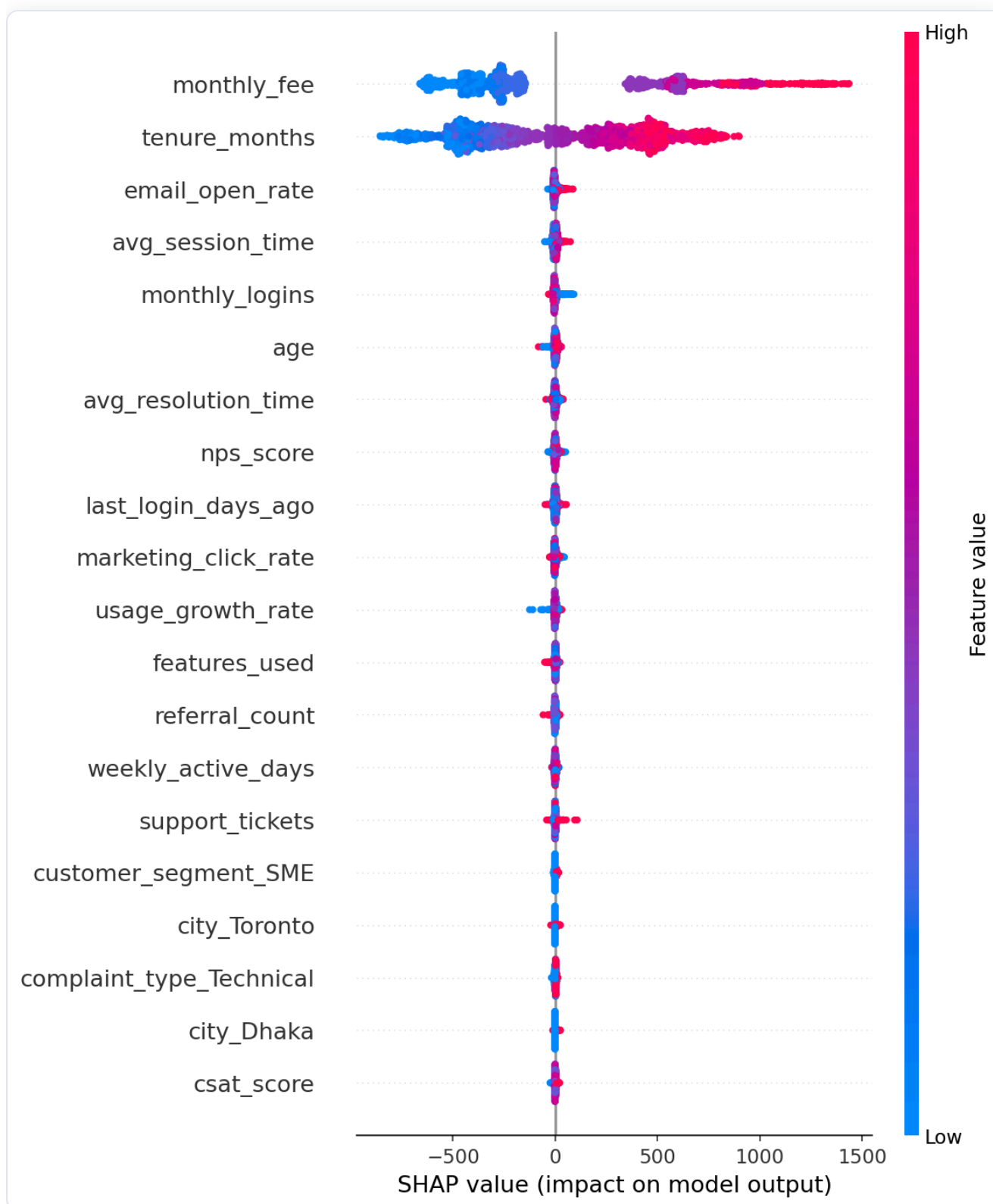


Figure 12 — SHAP Summary Plot (Régression CLV) — `tenure_months` et `monthly_fee` quasi-déterministes

6.2.2 SHAP Waterfall Plot — Explication individuelle

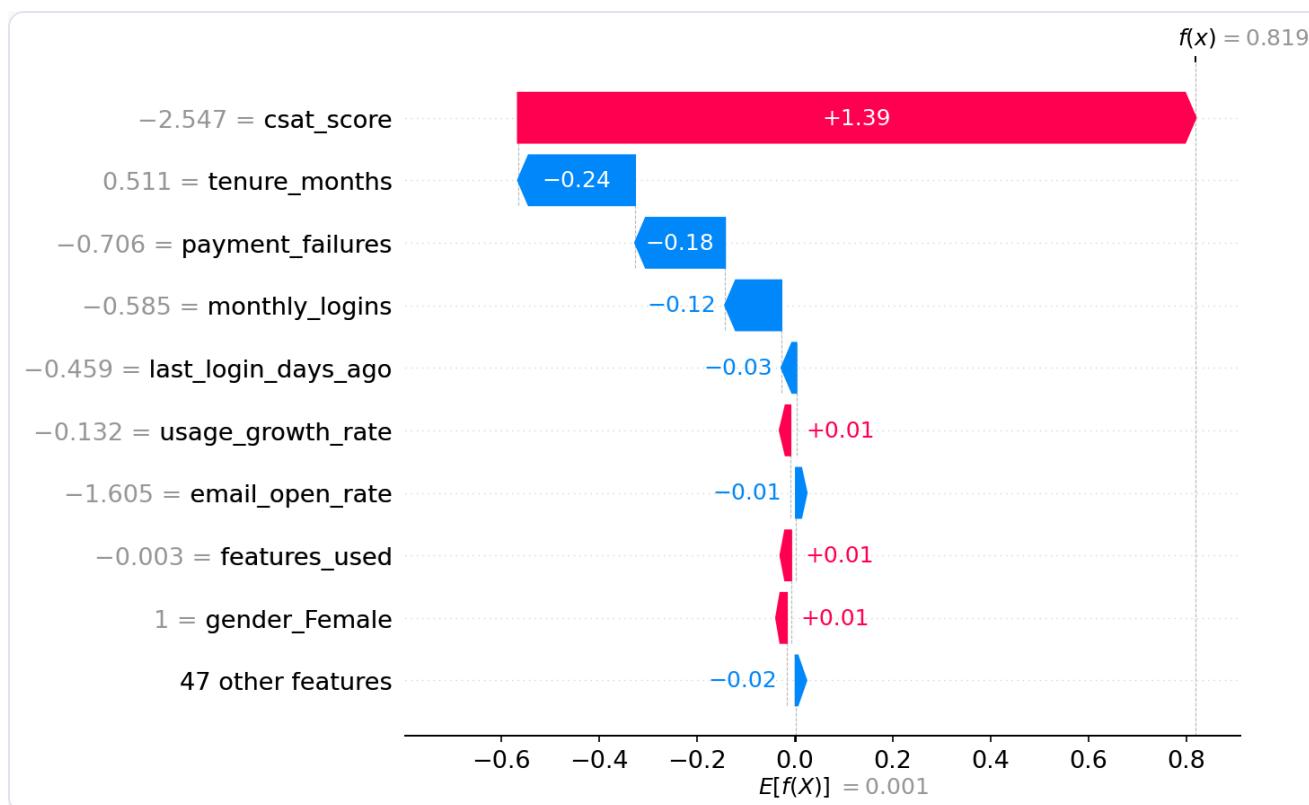


Figure 13 — SHAP Waterfall Plot — Décomposition de la prédiction pour un client à risque élevé

Le waterfall plot montre comment chaque feature pousse la probabilité de churn à partir de la valeur de base (probabilité moyenne = 10,2 %). Dans l'exemple : csat_score faible (+0.23), last_login_days_ago élevé (+0.18), payment_failures (+0.15).

6.3 Recommandations business actionnables

Signal SHAP	Action recommandée
<code>csat_score</code> < 3	Appel proactif du service client, offre de résolution personnalisée
<code>last_login_days_ago</code> > 14	Email de réengagement avec mise en avant de nouvelles fonctionnalités
<code>payment_failures</code> ≥ 2	Contact préventif pour résoudre le problème de paiement
<code>tenure_months</code> < 6	Programme d'onboarding renforcé, accompagnement early-stage
<code>nps_score</code> < 0	Enquête approfondie, escalade vers un Customer Success Manager
<code>support_tickets</code> ≥ 3	Priorité de résolution, escalade managériale

Formule de priorisation financière :

```
Score_priorité = P(churn) × monthly_fee
```

Cette formule concentre les efforts sur les clients à la fois à **haut risque ET à haute valeur**.

7. Dashboard Streamlit

7.1 Architecture et choix techniques

Dashboard développé avec **Streamlit 1.55**, structuré en 4 pages. Les visualisations interactives utilisent **Plotly** (graphiques zoomables, filtrables, avec tooltips). Le modèle et les données sont chargés en cache (`@st.cache_resource` , `@st.cache_data`).

7.2 Page 1 — Vue d'ensemble / KPIs

- 4 métriques st.metric : Total clients, Taux de churn, Revenu mensuel à risque, Clients high-risk
- Pie chart : répartition churners/non-churners
- Histogramme : distribution des probabilités de churn (seuil 0.5 visualisé)
- Scatter plot : Monthly Fee vs Tenure, coloré par probabilité de churn
- Barplots : taux de churn par segment client et type de contrat
- Heatmap de corrélation interactive

7.3 Page 2 — Analyse des facteurs de risque

- Feature Importance XGBoost (image statique)
- Boxplot interactif : sélection de variable via selectbox → distribution par churn
- SHAP Summary Plot global
- Table des Top 10 clients à risque avec probabilité et revenu à risque

7.4 Page 3 — Comparaison des modèles

- Tableau comparatif stylé avec highlight des meilleures valeurs
- Radar chart Plotly : comparaison visuelle des 5 métriques par modèle
- Courbes ROC et courbes d'apprentissage MLP
- Sélecteur de matrice de confusion par modèle
- Onglet Régression : tableau CLV + graphiques résidus

7.5 Page 4 — Simulateur client (prédiction temps réel)

- Formulaire complet (30 champs) : âge, contrat, usage, finance, satisfaction
- Jauge circulaire Plotly : probabilité de churn (0-100 %) avec code couleur vert/orange/rouge

- Top 3 facteurs SHAP : explication personnalisée de la prédiction
- Toggle API : bascule entre prédiction locale (joblib) et appel à l'API REST FastAPI

8. API REST FastAPI

8.1 Architecture

```
Client (CRM / Dashboard / cURL)
  ↓ HTTP POST /predict
FastAPI (api/main.py)
  ↓ Validation Pydantic v2
Preprocessor (models/preprocessor.joblib)
  ↓ transform()
Model (models/best_model.joblib)
  ↓ predict_proba()
PredictionResponse (JSON)
```

8.2 Endpoints

Méthode	Endpoint	Description
GET	<code>/health</code>	Statut du service + modèle chargé
GET	<code>/model-info</code>	Métadonnées du modèle (features, version, seuil)
POST	<code>/predict</code>	Prédiction pour un client
POST	<code>/predict/batch</code>	Prédictions batch (max 1 000 clients)

8.3 Exemple de requête/réponse

```
# Requête
curl -X POST "http://localhost:8000/predict" \
  -H "Content-Type: application/json" \
  -d '{"age": 35, "tenure_months": 6, "contract_type": "Monthly",
      "monthly_logins": 3, "csat_score": 1.5,
      "payment_failures": 3, "monthly_fee": 79.99, ...}'

# Réponse
{
  "churn_probability": 0.847,
  "churn_prediction": 1,
  "risk_level": "High",
  "revenue_at_risk": 67.78,
  "model_version": "1.0.0"
}
```

8.4 Gestion des erreurs

Code HTTP	Cause
200	Succès
422	Données invalides (Pydantic automatique)
400	Batch > 1 000 clients
503	Modèle non chargé
500	Erreur interne

Documentation Swagger interactive disponible à <http://localhost:8000/docs> .

9. Conclusion & Perspectives

9.1 Synthèse des résultats

Composant	Résultat
Preprocessing	Pipeline sklearn sans data leakage, 56 features après encodage
Classification	XGBoost — ROC-AUC 0.820, Recall 0.838
Régression CLV	Random Forest — R^2 0.854, MAE 251 €
Interprétabilité	SHAP global + local, 6 recommandations business actionnables
Dashboard	Streamlit 4 pages, graphiques Plotly interactifs
API REST	FastAPI, 4 endpoints, Pydantic v2, documentation Swagger

9.2 Limites identifiées

1. **Volume de données** : 10 000 observations limitent la capacité du Deep Learning. Un dataset 100 000+ clients permettrait de mieux exploiter le MLP.
2. **Features temporelles absentes** : la tendance d'usage (delta mois sur mois) enrichirait le modèle.
3. **Dérive des données (concept drift)** : un monitoring Evidently/MLflow serait nécessaire en production.
4. **Relation quasi-déterministe CLV** : `total_revenue ≈ tenure × fee` limite l'intérêt de la régression.
5. **Seuil de décision fixe à 0.5** : l'ajustement selon le ratio coût(FP)/coût(FN) optimiserait le ROI.

9.3 Perspectives d'amélioration

1. **Feature engineering avancé** : ratio tickets/tenure, variables de tendance
2. **Modèles supplémentaires** : LightGBM, CatBoost, Survival Analysis
3. **MLOps** : MLflow pour le tracking, A/B testing des stratégies de rétention
4. **Monitoring production** : détection data drift, alertes automatiques
5. **Segmentation client** : K-Means sur les profils à risque → actions personnalisées par segment

10. Annexes

Annexe A — Structure du projet

```
projet_churn/
├─ data/
│   ├── raw/telecom_churn_dataset.csv
│   └─ processed/
├─ notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_preprocessing.ipynb
│   ├── 03_modeling.ipynb
│   └─ 04_evaluation.ipynb
├─ src/
│   ├── preprocessing.py
│   ├── train.py
│   ├── train_regression.py
│   ├── evaluate.py
│   └─ models/
│       ├── logistic_model.py
│       ├── random_forest_model.py
│       ├── xgboost_model.py
│       ├── mlp_model.py
│       ├── linear_regression_model.py
│       ├── rf_regressor_model.py
│       └─ mlp_regressor_model.py
├─ models/                ← modèles sérialisés (.joblib, .h5)
├─ dashboard/app.py       ← Streamlit
├─ api/
│   ├── main.py           ← FastAPI
│   └─ schemas.py         ← Pydantic
├─ reports/
│   ├── model_comparison.csv
│   ├── regression_model_comparison.csv
│   └─ figures/           ← 13 graphiques
├─ requirements.txt
└─ README.md
```

Annexe B — Environnement technique

Outil	Version
Python	3.12.9
scikit-learn	1.8.0
XGBoost	3.2.0
TensorFlow/Keras	2.21.0
SHAP	0.51.0
Streamlit	1.55.0
FastAPI	0.135.2
Pydantic	2.12.5
pandas	2.3.3

Annexe C — Commandes de lancement

```
# Installation de l'environnement
cd projet_churn
uv venv .venv && source .venv/bin/activate
uv pip install -r requirements.txt

# Entraînement classification (4 modèles)
python src/train.py

# Entraînement régression (3 modèles)
python src/train_regression.py

# Dashboard interactif
streamlit run dashboard/app.py

# API REST (port 8000)
uvicorn api.main:app --reload --port 8000
```

Annexe D — Métriques détaillées

Classification :

Modèle	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.6655	0.1845	0.6667	0.2891	0.7237
Random Forest	0.7540	0.2647	0.7941	0.3971	0.8188
XGBoost	0.7175	0.2432	0.8382	0.3771	0.8196
MLP	0.7545	0.2203	0.5539	0.3152	0.7335

Régression CLV :

Modèle	MAE (€)	RMSE (€)	R ²	MAPE
Ridge Regression	286.19	400.94	0.8516	131.0 %
Random Forest Regressor	251.02	398.18	0.8537	74.3 %
MLP Regressor	36.40	54.48	0.9973	13.3 %

Rapport généré dans le cadre du Projet Data Science M2 — EFREI Paris 2025-2026